

Task 1: Dynamic Student Report Card Using Destructuring and Template Literals

Problem Statement:

You are given a student object that contains the details of a student including name, roll number, and marks in 5 subjects.

Using object and array destructuring and template literals, write a program to generate a formatted report card in the console.

Your program should:

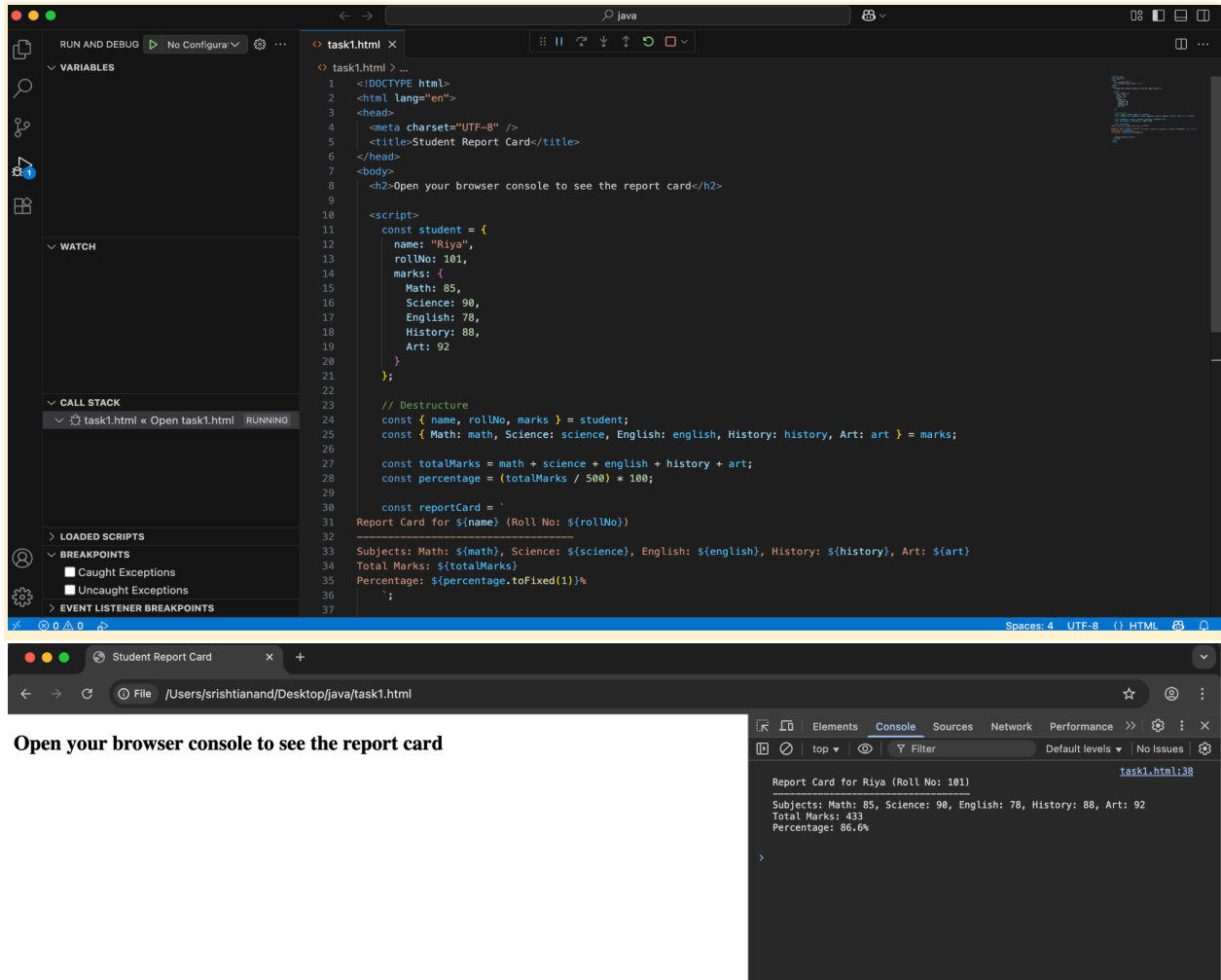
- Calculate the total and percentage.
- Use destructuring to extract the marks.
- Use template literals to display a multi-line summary like:

Report Card for Riya (Roll No: 101)

Subjects: Math: 85, Science: 90, English: 78, History: 88, Art: 92

Total Marks: 433

Percentage: 86.6%



Task 2: Build a Custom Greeting Generator with Default Parameters and Arrow Functions

Problem Statement:

- Write a function named `generateGreeting` that:
- Takes in name, role, and location as parameters (all with default values).
- Uses an arrow function to return a greeting like:

"Hello [name]! Your role is [role] and you are currently in [location]."

Requirements:

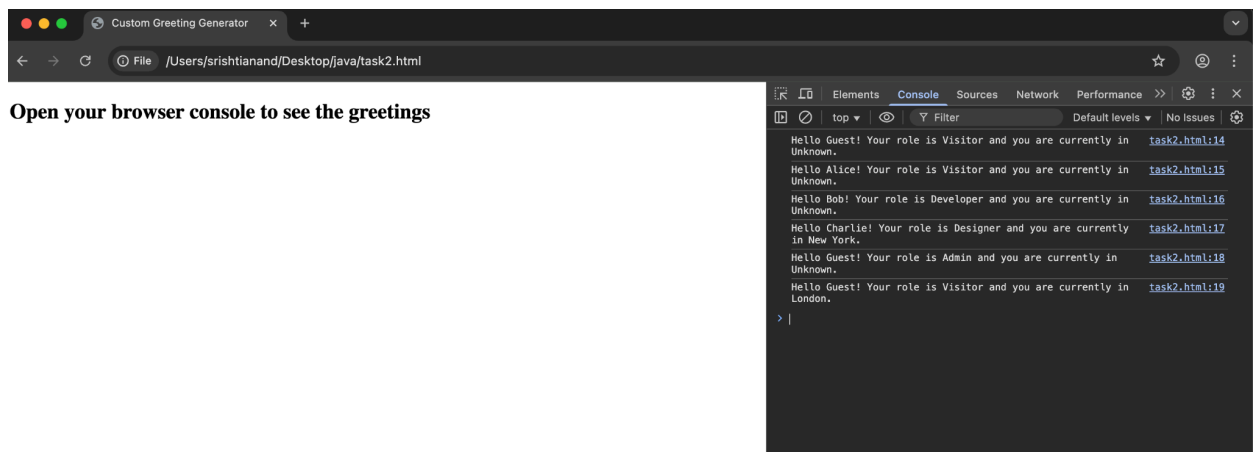
- Do not use function keyword.

- Use template literals for message construction.
- Call the function with different combinations of arguments and no arguments.
- Only one function should handle all variations without breaking.

```

1 <!DOCTYPE html>
2 <html lang="en">
3 <head>
4   <meta charset="UTF-8" />
5   <title>Custom Greeting Generator</title>
6 </head>
7 <body>
8   <h2>Open your browser console to see the greetings</h2>
9
10  <script>
11    const generateGreeting = (name = "Guest", role = "Visitor", location = "Unknown") =>
12      `Hello ${name}! Your role is ${role} and you are currently in ${location}.`;
13
14    console.log(generateGreeting());
15    console.log(generateGreeting("Alice"));
16    console.log(generateGreeting("Bob", "Developer"));
17    console.log(generateGreeting("Charlie", "Designer", "New York"));
18    console.log(generateGreeting(undefined, "Admin"));
19    console.log(generateGreeting(undefined, undefined, "London"));
20  </script>
21 </body>
22 </html>
23

```



Task 3: Implement a Data Type Analyzer (without typeof)

Problem Statement:

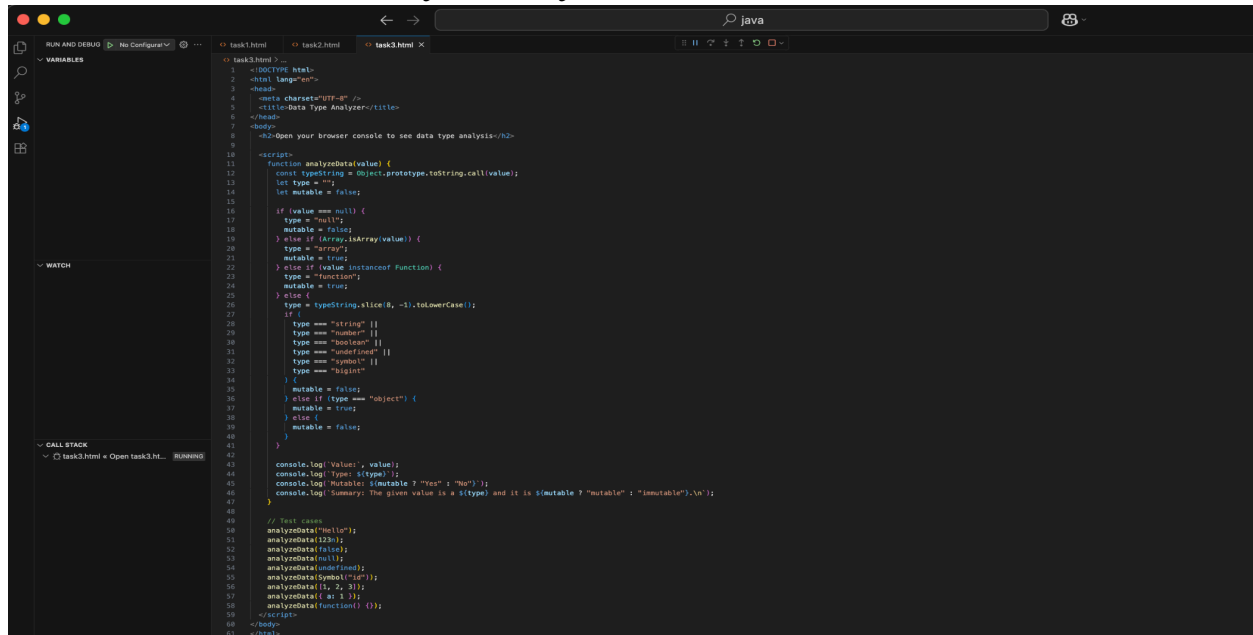
Create a function `analyzeData` that takes any input and prints:

- What kind of primitive or reference type it is (without using `typeof`).
- Whether it's mutable or immutable.
- A summary message.

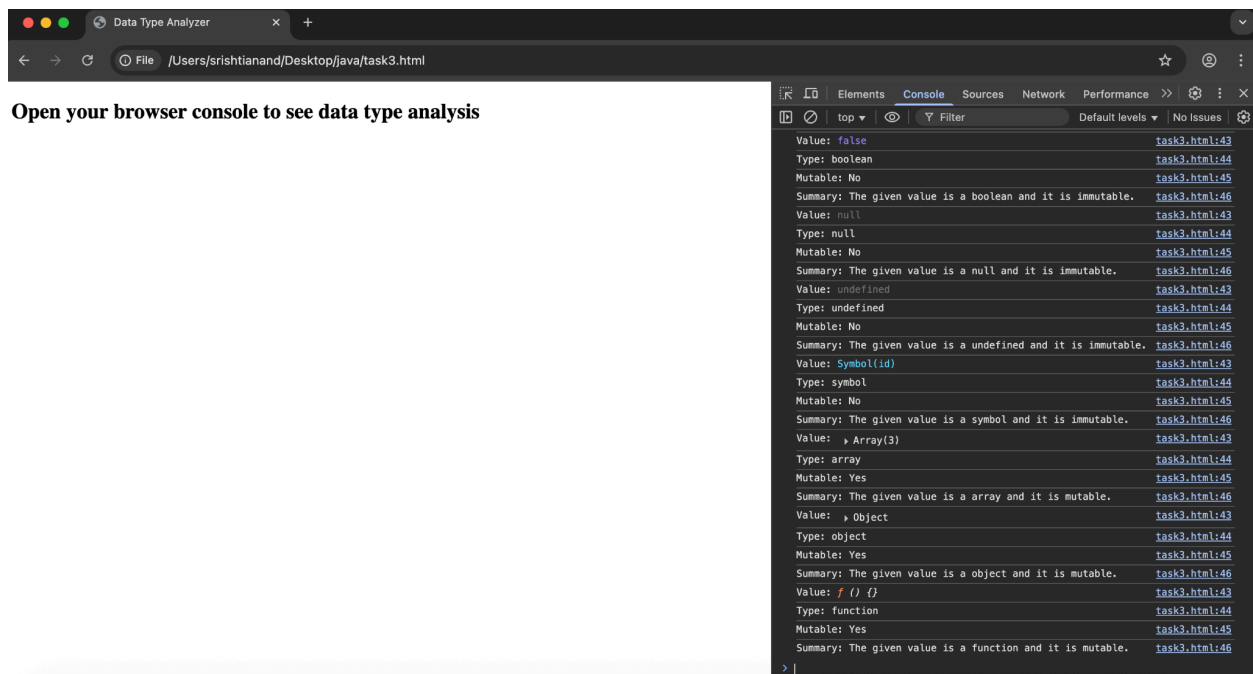
You cannot use typeof, but must still determine if it's:

- string, number, boolean, undefined, null, symbol, bigint
- array, object, function

You must figure out creative ways to determine the type, such as using constructor checks, Array.isArray, etc.



```
1 <!DOCTYPE html>
2 <html lang="en">
3 <head>
4 <meta charset="UTF-8" />
5 <title>Data Type Analyzer</title>
6 </head>
7 <body>
8 <!-- Open your browser console to see data type analysis -->
9
10 <script>
11 function analyzeData(value) {
12   console.log('Type: ' + typeof value);
13   let type = '';
14   let mutable = false;
15
16   if (value === null) {
17     type = 'null';
18     mutable = false;
19   } else if (Array.isArray(value)) {
20     type = 'array';
21     mutable = true;
22   } else if (value instanceof Function) {
23     type = 'function';
24     mutable = true;
25   } else {
26     type = typeof value.slice(0, -1).toLowerCase();
27     if (
28       type === 'string' ||
29       type === 'number' ||
30       type === 'boolean' ||
31       type === 'undefined' ||
32       type === 'symbol' ||
33       type === 'bigint'
34     ) {
35       mutable = false;
36     } else if (type === 'object') {
37       mutable = true;
38     } else {
39       mutable = false;
40     }
41   }
42
43   console.log('Value: ', value);
44   console.log('Type: ' + type);
45   console.log('Mutable: ' + (mutable ? 'Yes' : 'No'));
46   console.log('Summary: The given value is a ' + type + ' and it is ' + (mutable ? 'mutable' : 'immutable') + '.');
47 }
48
49 // Test cases
50 analyzeData('hello');
51 analyzeData(123);
52 analyzeData(false);
53 analyzeData(null);
54 analyzeData(undefined);
55 analyzeData(Symbol('id'));
56 analyzeData([1, 2, 3]);
57 analyzeData({ a: 1 });
58 analyzeData(function() {});
59 </script>
60 </body>
61 </html>
```



Task 4: Color Mixer using Destructuring and Spread/Rest Operators

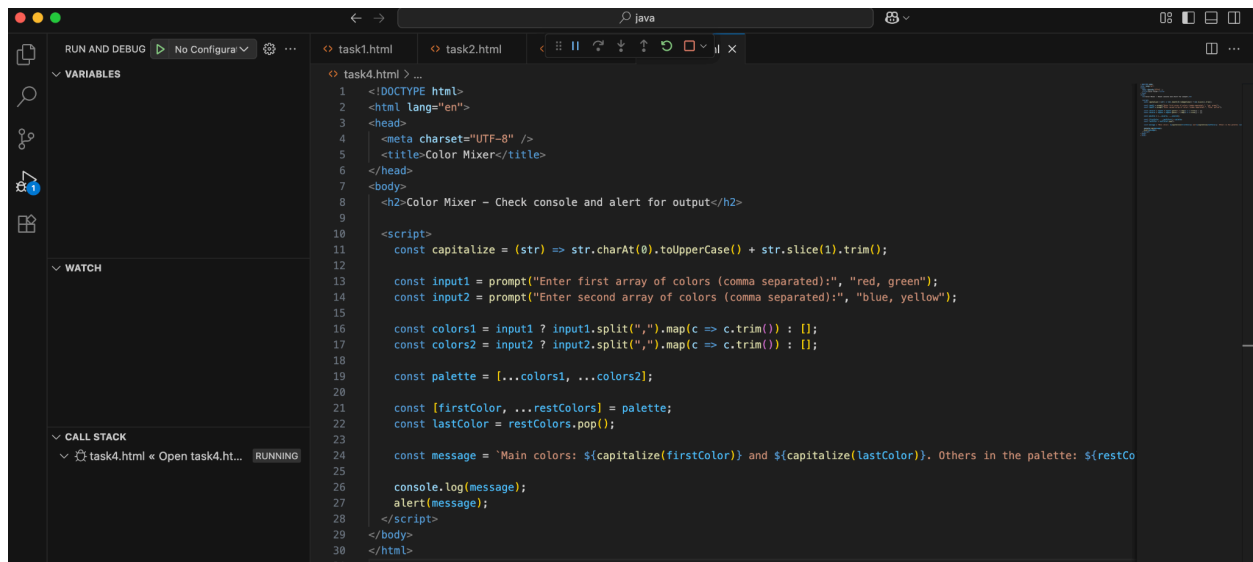
Problem Statement:

Write a program that:

- Takes two arrays of colors from the user (e.g., ["red", "green"] and ["blue", "yellow"]).
- Merges the arrays using the spread operator into a new array called palette.
- Extract the first and last colors from the palette using array destructuring.
- Collect the middle colors using the rest operator.
- Display a message like:

"Main colors: Red and Yellow. Others in the palette: Green, Blue."

You are not allowed to use loops or array indexing manually.



```
1 <!DOCTYPE html>
2 <html lang="en">
3 <head>
4   <meta charset="UTF-8" />
5   <title>Color Mixer</title>
6 </head>
7 <body>
8   <h2>Color Mixer - Check console and alert for output</h2>
9
10  <script>
11    const capitalize = (str) => str.charAt(0).toUpperCase() + str.slice(1).trim();
12
13    const input1 = prompt("Enter first array of colors (comma separated):", "red, green");
14    const input2 = prompt("Enter second array of colors (comma separated):", "blue, yellow");
15
16    const colors1 = input1 ? input1.split(",").map(c => c.trim()) : [];
17    const colors2 = input2 ? input2.split(",").map(c => c.trim()) : [];
18
19    const palette = [...colors1, ...colors2];
20
21    const [firstColor, ...restColors] = palette;
22    const lastColor = restColors.pop();
23
24    const message = `Main colors: ${capitalize(firstColor)} and ${capitalize(lastColor)}. Others in the palette: ${restColors.join(", ")}`;
25
26    console.log(message);
27    alert(message);
28  </script>
29 </body>
30 </html>
```

